ShieldScrape: Secure & Scalable Cloud-Based Web Data Collection

Aryan Mayur Dhuru
Computer Science
Indiana University Bloomington
Bloomington, Indiana, USA
adhuru@iu.edu

Suhana Ambol
Data Science
Indiana University Bloomington
Bloomington, Indiana, USA
sambol@iu.edu

Varunkumar Sonawane
Computer Science
Indiana University Bloomington
Bloomington, Indiana, USA
vsonawa@iu.edu

Shivali Mate
Computer Science
Indiana University Bloomington
Bloomington, Indiana, USA
shimate@iu.edu

I. Problam Statement

Shield Scrape aims to solve the challenge of manually tracking football league standings by automating the process of extracting and updating current standings from top leagues. With match data frequently changing, fans and analysts lack a consistent, real-time source for league positions. Shield Scrape addresses this by building an automated, cloud-native pipeline that scrapes, processes, and stores live standings every match day, delivering structured, up-to-date data across multiple leagues.

Shield Scrape will be resilient to web structure changes, compliant with scraping limitations, and scalable to support additional leagues and competitions. By integrating AWS services such as S3, Glue, DynamoDB, Lambda, and Fargate, we aim to build a maintainable and extensible solution that supports downstream visualization, analytics, and alerting systems.

II. Project Objectives

The primary goal of Shield Scrape is to create an automated system that reliably scrapes and processes real-time football standings from the top leagues around the world, including the English Premier League, La Liga, Serie A, and Bundesliga. These standings are to be extracted with high accuracy, structured consistently, and stored securely in a cloud environment for further processing and analysis. The project aims to reduce the dependency on manual scoreboard tracking and enable seamless integration with analytical tools and dashboards.

To achieve this, the system is built using a modular architecture powered by AWS services such as S3, Glue, DynamoDB, Lambda, and Fargate. The scraper is designed to run automatically on every match day at specific intervals, ensuring timely updates of league tables. The project also emphasizes parallel scraping capabilities using AWS Fargate, enabling simultaneous data extraction from multiple leagues without resource conflicts or bottlenecks.

By the end of the project, we expect to deliver a scalable and reusable scraping pipeline that can be extended to include additional sports or competitions. A user-friendly web dashboard will display live league standings and allow users to track Champions League qualification and relegation zones across different leagues. The outcome will serve as a foundation for building advanced sports analytics and fan engagement tools in the future.

III. Review of the Problem Being Addressed

In the modern sports analytics landscape, consistent and structured access to live and historical match statistics is crucial. However, football data is often spread across websites, making aggregation and daily analysis difficult. This project addresses the challenge of building a reliable data ingestion pipeline to extract and store structured information from publicly available sources.

The specific problem addressed is building an automated pipeline that scrapes English Premier League (EPL) standings and team statistics from the ESPN website and presents them in a format suitable for querying, analysis, and visualization. One major issue that arises in this domain is respecting scraping limitations imposed by websites via `robots.txt`, which may restrict automated access to parts of a website. Thus, identifying feasible scraping endpoints and formats is a foundational challenge.

IV. Technical Approach

Our technical approach includes the following components:

1) *Web Scraping via Scrapy:*
   We used the Scrapy Python framework to extract the EPL points table from ESPN. The scraper parses the team names, matches played, wins, draws, losses, goals scored, goals conceded, goal difference, and points
2) *Storage to AWS S3:*
   The extracted data is output in **.jsonl** format (JSON Lines) and uploaded directly to a structured path in an AWS S3 bucket. This format was chosen to simplify downstream ingestion by AWS Glue and ensure compatibility.
3) *Schema Definition using AWS Glue:*
   A Glue Crawler scans the JSONL file in S3, infers the schema, and creates a table in a defined Glue database (epl_db). This makes the raw data queryable via Athena and usable in ETL jobs.

4) *ETL Job using AWS Glue and PySpark:*
We created a Glue job using the script editor that loads the data from the Glue table and writes it into a DynamoDB table named epl_standings. This script dynamically reads the data, checks record counts, and pushes it to DynamoDB.
5) *IAM Role & Access Control:*
We created and attached a dedicated IAM role to our Glue job, granting only the required permissions (dynamodb: PutItem, BatchWriteItem, DescribeTable) to enforce fine-grained access control.
6) *Parallel Scraping with AWS Fargate:*
As we scale to multiple league pages (e.g., La Liga, Serie A, Bundesliga), we plan to use AWS Fargate to enable parallel and containerized scraping jobs. This serverless container management service allows us to run multiple Scrapy spiders concurrently without managing EC2 instances, thus optimizing scraping time and reliability.

V. Methodology

The methodology for Shield Scrape is centered on building a reliable, automated, and scalable pipeline for collecting and managing football standings data. We began by identifying trusted public sources like ESPN that publish live league tables in structured HTML formats. After confirming scraping feasibility through a review of the website's robots.txt and examining the structure of the HTML DOM, we developed a Scrapy spider to target key data points including team names, games played, wins, losses, goals, and points.

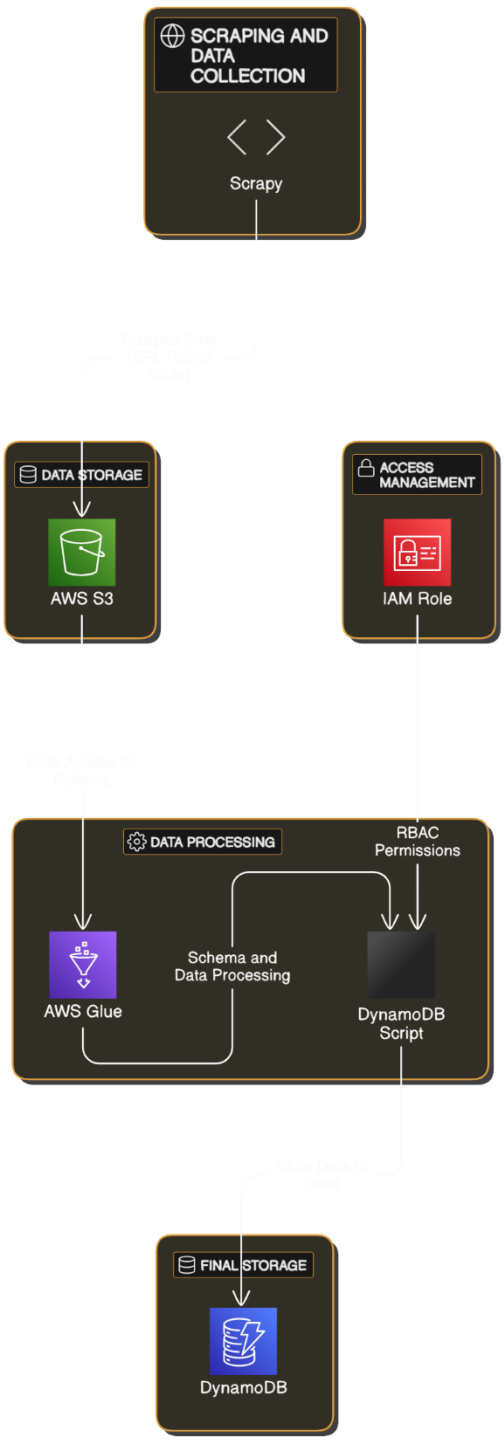
Once the data is extracted, it is stored in .jsonl (JSON Lines) format and uploaded directly to an Amazon S3 bucket. This choice of format simplifies downstream integration, particularly with AWS Glue, which uses crawlers to automatically infer the schema of the incoming data and generate a table in the Glue Data Catalog. Glue enables easy querying of S3-hosted data via Amazon Athena and also serves as the source for transformation jobs.

To convert the data into a queryable format for real-time applications, a Glue ETL job reads from the schema-defined Glue table and writes each record to a DynamoDB table, with team as the partition key. The entire process is designed to be repeatable and automated. We plan to integrate AWS Lambda and EventBridge to trigger this pipeline on match days, and AWS Fargate to enable parallel scraping of multiple leagues simultaneously. This methodology ensures accuracy, efficiency, and adaptability as the project scales to include more competitions.

- Data Source Identification: Analyzed websites for structure, feasibility, and **robots.txt** compliance.
- Scrapy Development: Used CSS selectors to parse nested HTML tables into structured records.
- Pipeline Automation: Configured AWS services to interconnect scraper output and database storage seamlessly.
- Data Modeling: Designed the DynamoDB schema with team as the partition key to support direct record access.
- Glue Schema Management: Managed schema evolution through automated crawling and re-indexing.

VI. Architecture

We adopted a modular design where each cloud service performs a dedicated role. S3 acts as the landing zone for structured files, Glue automates schema discovery and ETL, and DynamoDB is optimized for key-based lookups. Future support for a REST API and front-end dashboard enables scalability and broader access.



VII. Issues & Lessons Learned

A. *Initial Issues:*

- Initially, AWS Glue recognized the JSON output as a single array object rather than individual rows.
- The crawler failed to infer a row-based schema, breaking ETL operations.

B. *Resolution:*

- Switched from .**json** to **.jsonl** format, writing one JSON object per line.
- Re-ran the Glue Crawler to correctly infer schema with each JSON object representing a team

C. *Athena Verification:*

- Verified data correctness via Athena queries on the Glue table.
- Added logging in Glue jobs to verify row counts before writing to DynamoDB.

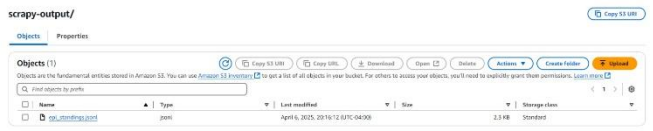
D. *Feasibility:*

- ESPN's standings page provided a clean, publicly available HTML table structure for scraping.
- Rate limits were not triggered, but we acknowledge potential access issues and are ready to switch sources if needed.

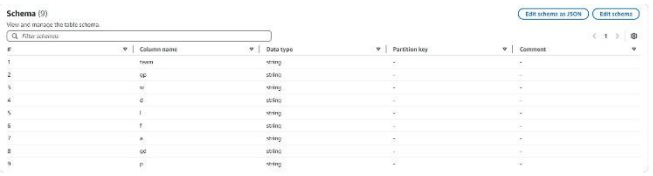
VIII. Preliminary Results

1) The EPL standings scraped using Scrapy were successfully stored in .jsonl format in the iu-sheild-scraper-bucket under the folder scrapy-output/.

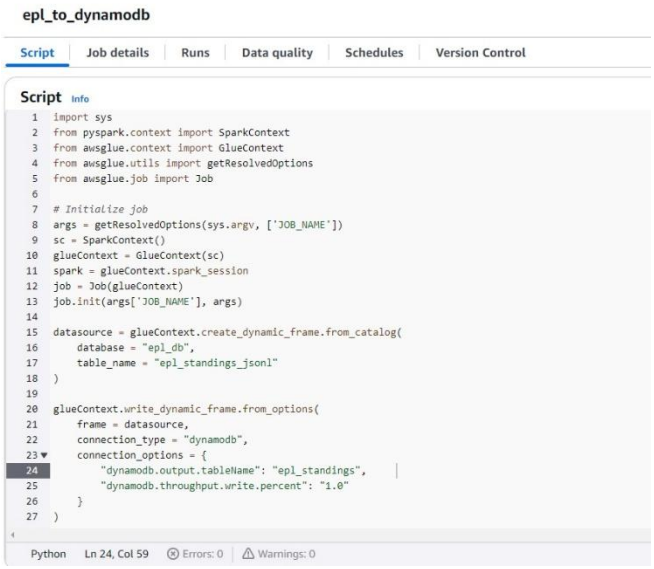
```
PS D:\Masters labs and projects\ECC\web scraper\SheildScrape> scrapy crawl jobspider -o
s3://iu-sheild-scraper-bucket/scrapy-output/epl_standings.jsonl
```



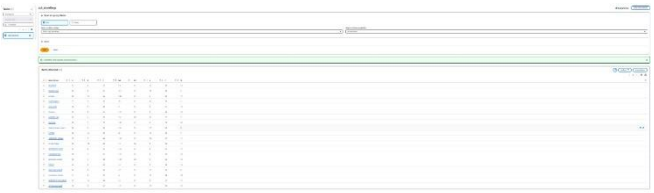
2) AWS Glue Crawler correctly inferred the schema of the JSONL file



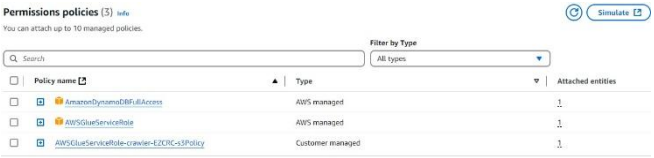
3) The job script used GlueContext to read from the Glue Catalog and write to DynamoDB using PySpark.



4) The final DynamoDB table view showed that all teams and their corresponding stats were correctly inserted.



5) IAM role attached to the Glue Job had the policy: AmazonDynamoDBFullAccess to allow writing to dynamodb



IX. Potential for Future Expansion

- Add multilingual support and expand to international competitions (e.g., UEFA, FIFA tournaments).
- Integrate machine learning models to predict match outcomes.
- Provide REST API access for developers to use the data in custom applications.
- Offer email or SMS alerts for users based on team performance thresholds.

X. Obstacles, Dependencies, and Backup Plan

- Obstacle: ESPN may update page structure or enforce stricter **robots.txt** restrictions.
- Backup: Switch to alternate open-data football APIs or sites (e.g., FootyStats, SofaScore).

- Dependency: AWS IAM misconfigurations or Glue service quotas can slow progress.The word “data” is plural, not singular.

XI. Expected Final Deliverables

- A complete pipeline: Scrapy → JSONL → S3 → Glue → DynamoDB → Dashboard
- Interactive web UI with real-time EPL standings & analytics
- Optional league selector and playoff zone filter
- Full documentation and architecture diagrams for end-to-end traceability

XII. Conclusion

Shield Scrape successfully delivers an automated solution to extract, process, and store live football standings using a cloud-native architecture. By leveraging Scrapy and AWS services like S3, Glue, and DynamoDB, we built a scalable pipeline that eliminates manual tracking and enables real-time data availability. With plans to expand to multiple leagues and integrate a UI, Shield Scrape sets the foundation for advanced sports analytics and fan engagement.

XIII. Roles of Team Members

1) Dhuru, Aryan Mayur -Cloud Infrastructure & Security: Manages AWS architecture, security policies, and fault tolerance mechanisms.

2) Ambol, Suhana - Dashboard & Visualization: Builds the frontend dashboard with AWS Amplify and analytics with QuickSight.

3) Mate, Shivali - Web Scraping & API Integration: Develops scraping logic with Scrapy/Puppeteer and integrates with API Gateway.

4) Sonawane, Varunkumar - Backend & Data Processing: Oversees ETL pipeline, AWS Glue, and data transformation logic.

References

[1] Scrapy Documentation: https://docs.scrapy.org/

[2] S3 Documentation - https://docs.aws.amazon.com/s3/

[3] AWS Glue Documentation - https://docs.aws.amazon.com/glue/

[4] Amazon DynamoDB Documentation - https://docs.aws.amazon.com/dynamodb/

[5] AWS Identity and Access Management Documentation - https://docs.aws.amazon.com/iam/

[6] AWS Security Best Practices: https://aws.amazon.com/whitepapers/aws-security-best-practices/